

Smart Metro Navigation Simulator: An Intelligent Route Planning and Ticket Booking System

A PROJECT REPORT

Submitted in partial fulfillment of the requirements of the course

DSA PLACEMENT BOOTCAMP:

MASTER DATA STRUCTURES AND ALGORITHMS FOR
INDUSTRY

Submitted By

Name : Dokka Pradeep Teja

Registration no : 12500472

Submitted To

Lovely Professional University

Phagwara, Punjab

2026

INDEX

S. No	Contents	Page No.
	Abstract	4
1	Chapter 1 – Introduction	6
1.1	Introduction	6
1.2	Background of the Study	7
1.3	Problem Statement	8
1.4	Objectives	9
1.5	Scope of the Project	11
1.6	Existing System	12
1.7	Proposed System	13
1.8	Technologies Used	15
1.9	System Architecture	17
1.10	Project Modules	20
1.11	Execution Flow	23
2	Chapter 2 – Algorithm	26
2.1	Overview of the Algorithm Module	26
2.2	Graph Representation	28
2.3	Graph Components	29
2.4	Node Class	30
2.5	Edge Class	31
2.6	Graph Class	33
2.7	Metro Network Builder	34
2.8	Dijkstra's Algorithm	35
2.9	Breadth-First Search (BFS)	37
2.10	Depth-First Search (DFS)	38

2.11	Priority Queue	39
2.12	Simulator : Live Train Delay Simulator	40
2.13	Web User Interface	42
2.14	Time Complexity Analysis	43
2.15	Space Complexity Analysis	45
3	Chapter 3 – Results	47
3.1	Introduction	47
3.2	Functional Results	48
3.3	Algorithm Performance	49
3.4	Observations	50
3.5	Advantages of the Developed System	51
3.6	Limitations	52
3.7	Chapter Summary	53
4	Chapter 4 – Conclusion	54
4.1	Conclusion	55
4.2	Future scope	56
	References	57

ABSTRACT

ABSTRACT

The Smart Metro Navigation Simulator is a web-based application developed to simulate intelligent metro route planning and passenger travel management in an urban metro network. The primary objective of the project is to demonstrate the practical application of Data Structures and Algorithms (DSA) in solving real-world public transportation problems. The system models a metro network as a graph in which metro stations are represented as nodes and railway connections between stations are represented as weighted edges. Using this graph-based representation, the simulator efficiently determines suitable routes and assists passengers in planning journeys based on travel time, ticket fare, and station changes.

The application implements several fundamental graph algorithms, including Dijkstra's Algorithm, Breadth-First Search (BFS), Depth-First Search (DFS), and Floyd-Warshall Algorithm. Dijkstra's Algorithm is used to identify the shortest and fastest route between two metro stations, while BFS is used to find routes with fewer station stops and DFS is used for graph traversal and network exploration. Floyd-Warshall Algorithm is used to calculate shortest paths between all station pairs in the metro network. The simulator also provides alternate routes and a live train delay simulation feature, which temporarily increases travel time on a selected metro connection and recalculates the most efficient available route.

The project follows a modular web-based design and is implemented using HTML, CSS, JavaScript, Node.js, and SQLite database technology. The application is organized into separate modules for metro network management, graph algorithms, delay simulation, route planning, user authentication, database handling, ticket booking, and graphical user interface. It provides an interactive user interface through which passengers can select starting and destination stations, compare route options, view the metro map, save journeys, book tickets, select coach preferences, and cancel tickets when required. User login and password reset functionality using email OTP authentication are also included to provide secure account access.

This project demonstrates how classical data structures such as graphs, queues, stacks, priority queues, and collections, together with efficient graph algorithms, can be integrated to develop an intelligent metro navigation system. The simulator provides users with an interactive environment for visualizing metro routes, comparing travel options, simulating train delays, and managing booked tickets. As a result, the project serves as both an educational tool for learning graph algorithms and a practical demonstration of their application in smart public transportation and metro passenger management systems.

Keywords: Smart Metro Navigation, Metro Route Planning, Graph Data Structure, Dijkstra's Algorithm, Breadth-First Search, Depth-First Search, Floyd–Warshall Algorithm, Route Optimization, Live Train Delay Simulation, Alternate Routes, Ticket Booking, User Authentication, SQLite Database, Data Structures and Algorithms.

CHAPTER 1

INTRODUCTION

1.1 Introduction

In recent years, the rapid growth of urban populations has increased the need for efficient, reliable, and user-friendly public transportation systems. Metro rail networks are widely used in modern cities because they provide fast, economical, and environmentally friendly travel. However, passengers often face difficulties in selecting the most suitable route because a metro journey may involve multiple stations, line changes, different ticket fares, alternate paths, and unexpected train delays. Traditional route planning methods may not provide clear information about the fastest route, cheapest ticket, or route with the fewest station changes. Therefore, intelligent metro navigation systems have become an important part of smart transportation management.

The Smart Metro Navigation Simulator is a web-based application developed to simulate metro route planning by applying fundamental concepts of Data Structures and Algorithms

(DSA). The project represents a metro network as a graph where metro stations such as Miyapur, Ameerpet, Secunderabad, MGBS, Nagole, and Malakpet are represented as nodes, while the railway connections between these stations are represented as weighted edges. Each connection stores information such as travel time, ticket fare, and metro line. This graph-based representation enables efficient route computation, network traversal, and route comparison using well-known graph algorithms.

The primary objective of this project is to demonstrate how graph algorithms can be applied to solve practical metro navigation problems. The simulator allows users to select a starting station and destination station, visualize the metro network, identify the fastest route, compare the cheapest route, find routes with fewer line changes, and view alternate routes. By integrating Dijkstra's Algorithm, Breadth-First Search (BFS), Depth-First Search (DFS), and Floyd-Warshall Algorithm, the application provides an interactive platform for understanding the role of graph theory in public transportation and route optimization. The project also includes a live train delay simulator that recalculates journey time and suggests a better route when a selected metro connection is delayed.

The project has been developed using HTML, CSS, JavaScript, Node.js, and SQLite database technology. The application is organized into multiple modules responsible for metro network creation, graph algorithm implementation, route computation, delay simulation, user authentication, password reset through email OTP, ticket booking, ticket cancellation, and graphical visualization. This modular design improves the maintainability, readability, security, and future scalability of the system.

From the perspective of the DSA Placement Bootcamp, this project serves as a practical implementation of important data structures and algorithms. Instead of studying graph algorithms only through theoretical examples, the Smart Metro Navigation Simulator demonstrates how these concepts can be combined to solve real-world challenges involving shortest path computation, graph traversal, alternate route discovery, dynamic delay handling, and ticket management. Consequently, the project bridges the gap between academic learning and practical software development in the field of smart transportation systems.

1.2 Background of the Study

The rapid growth of urban populations and public transportation networks has increased the complexity of daily travel planning. Metro passengers need to make quick decisions about the best route, travel time, ticket fare, and station changes. In large metro networks, manually identifying a suitable route can be difficult, especially when multiple metro lines are available or when a train delay affects a connection. Delays in route planning may lead to unnecessary waiting time, increased travel cost, missed connections, and passenger inconvenience. Therefore, efficient route planning has become an essential component of modern smart transportation systems.

One of the most effective ways to model a metro transportation network is by using a graph data structure. In graph theory, metro stations are represented as nodes or vertices, while railway connections between stations are represented as edges. Each edge can store information such as travel time, ticket fare, and metro line. This representation allows graph algorithms to efficiently analyze connectivity, determine optimal routes, identify alternate paths, and simulate changes in the metro network.

The Smart Metro Navigation Simulator utilizes this graph-based approach to create a virtual metro network. The simulator enables users to select a starting station and destination station, calculate the fastest route, find the cheapest ticket route, identify routes with fewer line changes, and view alternate route options. It also provides a live train delay simulation feature, where a delay can be added to a selected metro connection and the system recalculates the route based on the updated travel time. This provides a practical demonstration of how graph algorithms can solve real-world transportation and route-planning problems.

The project has been developed as part of the DSA Placement Bootcamp: Master Data Structures and Algorithms for Industry, with the objective of applying theoretical DSA concepts in a practical software application. Instead of implementing algorithms independently, the project integrates Dijkstra's Algorithm, Breadth-First Search, Depth-First Search, and Floyd-Warshall Algorithm into a complete metro navigation system. These algorithms interact with the metro graph to solve different routing requirements, such as

shortest travel time, minimum station stops, graph traversal, and all-pairs shortest path calculation.

The application also demonstrates the importance of modular software development and web application design. Different functionalities such as metro network management, route computation, delay simulation, user authentication, database handling, password reset through email OTP, ticket booking, and ticket cancellation are implemented in separate modules. This modular architecture improves code readability, simplifies maintenance, and allows future enhancements without affecting the overall system.

Overall, the project illustrates how data structures and algorithms are not limited to academic exercises but play a vital role in designing intelligent systems capable of supporting smart transportation, passenger decision-making, and efficient metro journey management.

1.3 Problem Statement

Metro passengers are required to make quick and accurate decisions regarding their daily travel routes. In a large metro network, multiple stations and metro lines connect different parts of the city, and passengers may have several possible routes to reach the same destination. Selecting an inappropriate route can increase travel time, ticket cost, waiting time, and the number of line changes. Unexpected train delays can also affect the planned journey and cause passenger inconvenience.

Conventional metro route planning methods often rely on static maps, manual route selection, or limited information about station connectivity. Such methods may not clearly identify the fastest route, cheapest ticket route, route with the fewest line changes, or suitable alternate routes. Additionally, when a train connection is delayed, passengers may find it difficult to determine whether they should wait for the delayed train or choose another available route.

The problem addressed in this project is the development of a smart metro navigation simulation system capable of representing a metro network, calculating efficient routes,

comparing travel options, simulating train delays, and managing passenger ticket bookings using graph algorithms. The system should provide an intuitive web-based interface that allows users to select stations, visualize the metro map, view route details, simulate delays, identify alternate routes, create user accounts, save journeys, and book metro tickets.

By combining graph data structures with shortest path and traversal algorithms, the Smart Metro Navigation Simulator aims to provide an efficient, interactive, and educational solution for understanding intelligent metro route planning and smart public transportation management.

1.4 Objectives

The primary objective of the Smart Metro Navigation Simulator is to develop an interactive web-based application that demonstrates the practical implementation of Data Structures and Algorithms in solving metro route planning and passenger management problems. The project aims to combine graph theory, shortest path algorithms, graph traversal techniques, web development, database management, and user authentication into a single application capable of simulating real-world metro travel scenarios.

The specific objectives of the project are as follows:

- To represent a metro transportation network using the Graph Data Structure, where metro stations are represented as nodes and metro railway connections are represented as edges.

- To implement Dijkstra's Algorithm for identifying the fastest route between two metro stations based on total travel time.
- To implement a fare-based route calculation for identifying the cheapest ticket route between selected stations.
- To implement Breadth-First Search (BFS) for finding routes with fewer station stops and analyzing metro network connectivity.
- To implement Depth-First Search (DFS) for exploring different paths within the metro network.
- To implement Floyd–Warshall Algorithm for calculating shortest paths between all pairs of metro stations.
- To simulate live train delays by dynamically increasing the travel time of a selected metro connection and observing its impact on the recommended route.
- To provide alternate route suggestions when the primary route is delayed, unavailable, or less suitable for the selected travel priority.
- To develop an interactive web-based graphical user interface using HTML, CSS, and JavaScript that enables users to visualize the metro map, select stations, compare routes, and simulate delays.
- To implement user registration, login, secure password reset through email OTP authentication, and SQLite database connectivity.

- To provide ticket booking functionality that allows users to save journeys, choose a travel date and coach preference, view a digital metro ticket, and cancel booked tickets.
- To demonstrate the practical application of graph algorithms, database management, and web development concepts in solving real-world smart transportation problems.
- To provide an educational platform for understanding the importance of Data Structures and Algorithms in software engineering, route optimization, and intelligent public transportation systems.
- To design a modular and scalable application that can be enhanced with real metro data, GPS tracking, online payments, and live train information in future versions.

1.5 Scope of the Project

The scope of the Smart Metro Navigation Simulator is limited to the simulation of metro route planning, passenger journey management, and ticket booking within a virtual metro network. The application focuses on demonstrating the practical use of graph-based algorithms for route optimization rather than providing a real-time official metro navigation service.

The simulator allows users to select and visualize a metro network consisting of interconnected stations and railway connections. It supports route planning based on multiple priorities, including fastest travel time, cheapest ticket fare, and fewest line changes. The

system also provides alternate route suggestions and simulates live train delays by modifying the travel time of a selected metro connection and recalculating the recommended route.

The project includes user registration, login, secure password reset through email OTP authentication, saved journey management, ticket booking, coach preference selection, digital ticket display, and ticket cancellation. User account information, saved journeys, and booked tickets are stored in a SQLite database. The application is designed primarily for educational purposes as part of the DSA Placement Bootcamp, enabling students to understand how graph data structures, graph algorithms, web development, and database technologies can be integrated into a practical software system.

The project provides an interactive environment for studying shortest path computation, graph traversal, route comparison, alternate route discovery, delay handling, user authentication, and ticket management through a web-based graphical interface. It demonstrates Dijkstra's Algorithm, Breadth-First Search, Depth-First Search, and Floyd-Warshall Algorithm using a metro transportation use case.

Although the current implementation successfully demonstrates core metro navigation concepts, it does not incorporate official real-time metro data, GPS-based passenger tracking, live train locations, online payment gateway integration, actual ticket validation, artificial intelligence-based demand prediction, or direct integration with government metro services. These features are beyond the scope of the present project and may be incorporated in future enhancements.

Overall, the project serves as a proof of concept for an intelligent metro route planning and passenger management system while emphasizing the practical implementation of Data Structures and Algorithms in solving real-world smart transportation problems.

1.6Existing System

Existing metro route-planning methods used by many passengers primarily rely on static metro maps, printed route charts, basic mobile maps, or manual route selection. These approaches may provide station information, but they often do not clearly compare the fastest route, cheapest ticket route, route with the fewest line changes, or alternate routes. Passengers

may need to manually check stations and metro lines before selecting a journey, which can be time-consuming and confusing in a large metro network.

Many traditional route-planning systems also lack interactive delay simulation capabilities. They do not allow users to temporarily add a train delay to a metro connection and observe how the recommended route changes. Furthermore, some simple navigation systems focus only on shortest distance or travel time and do not provide fare comparison, line-change analysis, alternate route suggestions, digital ticket booking, user login, saved journey management, or ticket cancellation features.

Another limitation of existing educational projects is that they usually demonstrate graph algorithms individually rather than integrating them into a complete transportation application. Students may understand Dijkstra's Algorithm, BFS, DFS, or Floyd-Warshall Algorithm theoretically, but they may have limited understanding of how these algorithms can work together in a real metro navigation system.

These limitations highlight the need for an intelligent metro navigation simulator capable of dynamically modeling metro routes, calculating multiple journey options, simulating train delays, managing user accounts, and demonstrating the practical application of graph algorithms in an integrated environment.

1.7Proposed System

The proposed system is an interactive web-based application named Smart Metro Navigation Simulator, developed using HTML, CSS, JavaScript, Node.js, and SQLite database technology. Unlike conventional metro route-planning methods that rely on static maps or manual route selection, the proposed system dynamically models a metro transportation network and computes efficient routes using graph algorithms.

The system represents the metro network as a graph, where each metro station is represented as a node and every railway connection between two stations is represented as an edge. Each connection stores information such as travel time, ticket fare, and metro line. This graph-based model enables efficient implementation of graph algorithms for route planning

and network traversal. Whenever a train delay is simulated, the system modifies the travel time of the selected connection. Based on the updated condition, the routing algorithms automatically determine the most suitable available route.

One of the major strengths of the proposed system is the integration of multiple graph algorithms within a single application. Dijkstra's Algorithm is used to compute the fastest route between two stations, while a fare-based route calculation is used to identify the cheapest ticket route. Breadth-First Search (BFS) is used to find routes with fewer station stops, Depth-First Search (DFS) is used to explore the metro network, and Floyd-Warshall Algorithm is used to calculate shortest paths between all pairs of stations. The system also provides alternate route suggestions and line-change analysis for better passenger decision-making.

The application provides a graphical web interface that allows users to interact with the simulator without requiring programming knowledge. Users can visualize the metro network map, select starting and destination stations, compare route options, simulate train delays, view route details, save journeys, and book tickets. The application also supports user registration, login, password reset through email OTP authentication, coach preference selection, digital ticket display, and ticket cancellation. The graphical interface improves user experience by presenting route information, travel time, ticket fare, and booking details in an intuitive manner.

The proposed system follows a modular software architecture in which different functionalities are organized into separate modules responsible for metro network management, graph algorithms, route planning, delay simulation, user authentication, database handling, ticket booking, and graphical user interface. This modular design improves code readability, simplifies maintenance, and allows new stations, algorithms, or advanced transportation features to be added with minimal modifications to the existing system.

Overall, the proposed system demonstrates how Data Structures and Algorithms can be integrated into a practical software application to solve metro route-planning problems while providing an educational platform for learning graph algorithms through visualization, simulation, and passenger journey management.

1.8 Technologies Used

Technology	Purpose
HTML5	Structure and layout of the web application
CSS3	Styling, responsive design, metro map, ticket, and user interface design
JavaScript	Client-side functionality, graph representation, route planning, and user interaction
Node.js	Server-side application development and API handling
SQLite	Storage of user accounts, saved journeys, OTP records, and ticket bookings
Graph Data Structure	Representation of metro stations and railway connections
Dijkstra's Algorithm	Fastest route calculation based on travel time
Breadth-FirstSearch (BFS)	Route calculation with fewer station stops and graph traversal
Depth-FirstSearch (DFS)	Metro network exploration and graph traversal
Floyd–Warshall Algorithm	Shortest path calculation between all pairs of metro stations
HTML SVG	Interactive visual representation of the metro network map

Node.js Crypto Module Password hashing, session token generation, and OTP security

Nodemailer and Gmail Sending password-reset OTP emails

SMTP

REST API Communication between the web interface, server, and database

Modular Programming Separation of route planning, authentication, booking, database, and UI logic

The Smart Metro Navigation Simulator has been developed using HTML, CSS, and JavaScript for the client-side interface, along with Node.js for server-side processing. JavaScript provides the flexibility required to implement graph algorithms, manage user interactions, simulate train delays, process route calculations, and update the metro map dynamically within a web browser.

The graphical user interface has been developed using HTML5 and CSS3. HTML is used to define the structure of the application, including station selectors, route result cards, login forms, ticket-booking forms, and the metro map section. CSS is used to create an attractive, responsive, and user-friendly interface with buttons, cards, coloured metro lines, digital tickets, and mobile-friendly layouts. JavaScript event handling enables users to interact with the simulator by selecting stations, changing route preferences, simulating delays, booking tickets, and managing their accounts.

The project primarily utilizes the Graph Data Structure to model the metro transportation network. Each metro station is represented as a node, while railway connections between stations are represented as edges. Each edge stores important information such as travel time, ticket fare, and metro line. This representation enables efficient implementation of graph algorithms for route computation, alternate route discovery, train delay simulation, and network visualization.

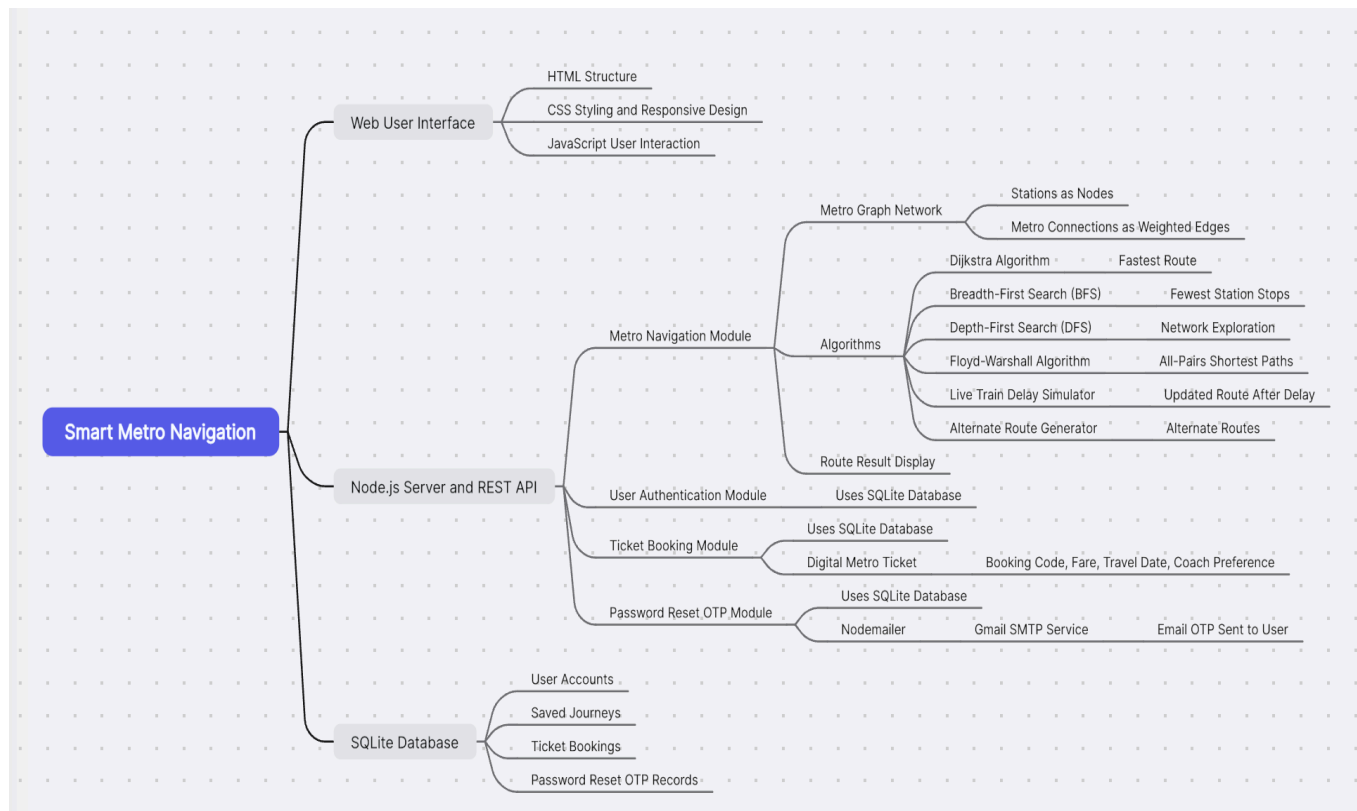
The routing functionality is achieved using Dijkstra's Algorithm, which determines the fastest route between two metro stations based on total travel time. A fare-based route calculation is

used to identify the cheapest ticket route. In addition, Breadth-First Search (BFS) and Depth-First Search (DFS) are implemented to demonstrate different graph traversal techniques. Floyd–Warshall Algorithm is used to calculate shortest paths between all pairs of metro stations, making it useful for comparing routes throughout the metro network.

The server-side application is developed using Node.js, which manages user registration, login, password reset, ticket booking, ticket cancellation, and database operations. SQLite is used as the local database for storing user information, saved journeys, password-reset OTP records, and booked ticket details. Passwords are stored securely using salted hashing through the Node.js Crypto Module. Nodemailer and Gmail SMTP are used to send secure six-digit OTP emails for password reset functionality.

The entire application follows a modular programming approach by organizing functionality into separate components for metro network management, graph algorithms, delay simulation, user authentication, database handling, ticket booking, and user interface design. This structure improves code readability, simplifies maintenance, and allows future enhancements to be added efficiently.

1.9System Architecture



The Smart Metro Navigation Simulator follows a modular web-based software architecture in which the user interface, metro network management, graph algorithms, delay simulation, user authentication, database handling, and ticket booking functionalities are implemented as independent components. This modular approach improves code readability, maintainability, security, and scalability by assigning a specific responsibility to each module.

The interaction begins with the web-based graphical user interface, where users perform various operations such as selecting starting and destination stations, choosing travel preferences, simulating train delays, comparing routes, creating accounts, saving journeys, and booking metro tickets. The interface is developed using HTML, CSS, and JavaScript and serves as the communication layer between the user and the internal processing modules.

When the user searches for a metro route, the request is forwarded to the Metro Navigation Module. This module processes the selected stations and travel priority, such as fastest route, cheapest ticket, or fewest line changes. When the user activates the live delay simulator, the module updates the travel time of the selected metro connection and recalculates the route based on the changed network condition.

The Metro Graph Management Module is responsible for maintaining the metro transportation network. It stores information about metro stations and railway connections using a graph data structure. Stations are represented as nodes, while metro connections are represented as weighted edges containing travel time, ticket fare, and metro line information. This module provides the routing algorithms with the necessary data required for shortest-path computation, graph traversal, alternate-route discovery, and metro map visualization.

The Algorithms Module forms the core of the application. It contains implementations of Dijkstra's Algorithm, Breadth-First Search (BFS), Depth-First Search (DFS), and Floyd-Warshall Algorithm. These algorithms process the metro graph to determine the fastest route, route with fewer station stops, metro network exploration path, and shortest paths between all pairs of stations. The module also supports alternate-route generation by evaluating different available connections in the metro network.

The User Authentication and Database Module manages user registration, login, password security, saved journeys, ticket details, and cancellation records. Node.js handles server-side requests and REST APIs, while SQLite stores user accounts, journey records, booking information, and OTP reset details. Password reset functionality uses Nodemailer and Gmail SMTP to send secure email OTPs to users.

Finally, the computed routes, delay effects, booking confirmations, and digital ticket details are returned to the web interface, where users can visualize metro paths, travel time, fare, line changes, alternate routes, and ticket information. This layered interaction between modules ensures that each component performs a dedicated function while collectively providing an efficient smart metro navigation and passenger management system.

1.10 Project Modules

The Smart Metro Navigation Simulator has been developed using a modular architecture in which related functionalities are grouped into separate modules. Each module performs a specific task and collaborates with other modules to complete the metro route planning, user management, and ticket booking process. This modular organization improves software maintainability, readability, security, and scalability.

Module 1: User Interface Module

This module provides the web-based graphical interface through which users interact with the application. It is developed using HTML, CSS, and JavaScript and consists of route-planning controls, station selection menus, route-priority buttons, delay simulation controls, metro map visualization, login forms, ticket-booking forms, and result displays. The user interface receives user input and communicates with the backend and algorithm modules to perform different operations.

Module 2: Metro Graph Management Module

The metro graph management module represents the metro transportation network. It stores all metro stations as nodes and all railway connections as weighted edges. Each connection includes information such as travel time, ticket fare, and metro line. This module is responsible for creating, maintaining, and updating the metro graph whenever a train delay is simulated or a route is calculated.

Module 3: Algorithm Module

The algorithm module implements the core graph algorithms used by the simulator. It includes Dijkstra's Algorithm for fastest-route calculation, a fare-based route calculation for cheapest ticket selection, Breadth-First Search (BFS) for finding routes with fewer station stops, Depth-First Search (DFS) for metro network exploration, and Floyd–Warshall Algorithm for shortest-path calculation between all station pairs. This module performs route computation, network analysis, and route comparison operations.

Module 4: Live Delay Simulation and Alternate Route Module

This module simulates live train delays by temporarily increasing the travel time of a selected metro connection. It communicates with the graph management and algorithm modules to recalculate the

recommended route according to the updated travel time. The module also identifies alternate routes by evaluating different available metro connections and provides passengers with route options when the primary route is delayed or less suitable.

Module 5: User Authentication and Database Module

The user authentication and database module manages user registration, login, logout, password hashing, session handling, saved journeys, and database records. It is developed using Node.js and SQLite database technology. User information, saved route details, ticket bookings, and password-reset OTP records are stored securely in the SQLite database. The module ensures that only authenticated users can save journeys, book tickets, view their ticket history, or cancel a ticket.

Module 6: Password Reset and Email OTP Module

This module provides secure password recovery functionality. When a user selects the forgot-password option, the system generates a six-digit OTP and sends it to the registered email address using Nodemailer and Gmail SMTP. The OTP has a limited validity period and a restricted number of verification attempts. After successful OTP verification, the user can create a new password, which is stored securely using password hashing.

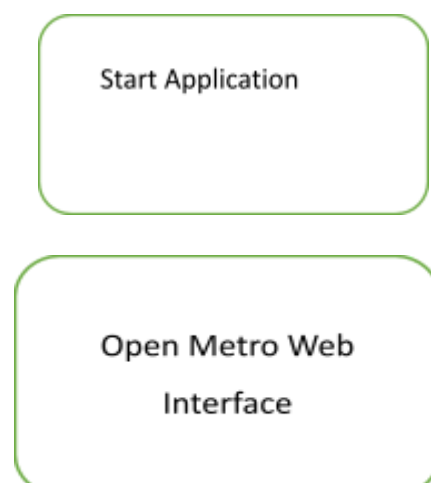
Module 7: Ticket Booking and Passenger Management Module

This module allows authenticated users to book metro tickets for their planned journey. It stores passenger name, travel date, coach preference, selected route, travel time, fare, and line-change information. After successful booking, the system generates a unique booking code and displays a digital metro ticket with route and passenger details. Users can also view their booked tickets and cancel a ticket when required.

Module 8: Data Model and Utility Module

This module defines the core data structures and helper functions used throughout the application. It manages metro station names, route details, travel-time values, fare values, metro line colours, booking codes, OTP expiry time, session tokens, and database validation. These utilities improve code readability, reduce duplication, and help prevent invalid input or incorrect data processing.

1.11 Execution Flow



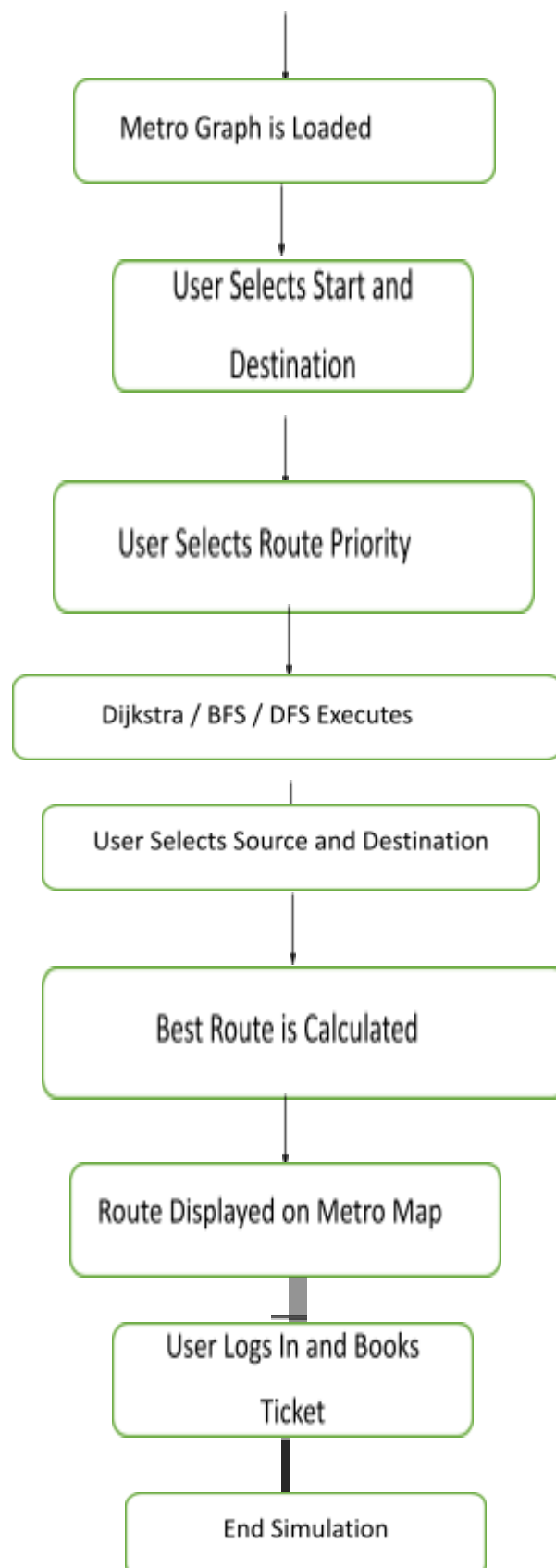


Figure 1.2: Execution Flow of the Smart metro navigation simulator

Execution Flow Description

The execution of the Smart Metro Navigation Simulator begins when the user opens the application through a web browser. The web server is started using Node.js, and the application interface is loaded through the localhost address. The web interface acts as the central platform through which users interact with the metro navigation system.

After the interface is initialized, the application loads the virtual metro network using the graph management module. Different metro stations are created as graph nodes, while railway connections between stations are represented as weighted graph edges. Each edge stores details such as travel time, ticket fare, and metro line. This metro graph serves as the foundation for all route-planning, delay-simulation, and network-analysis operations performed by the application.

The user then selects the starting station and destination station from the available metro stations. The user can also select a travel priority such as fastest route, cheapest ticket route, or route with the fewest line changes. These selected inputs are forwarded to the metro navigation and algorithm modules for processing.

Based on the requested operation, the application executes the appropriate graph algorithm. Dijkstra's Algorithm is used to calculate the fastest route based on travel time, while the fare-based route calculation identifies the cheapest ticket route. Breadth-First Search (BFS) is used to find routes with fewer station stops, Depth-First Search (DFS) is used for metro network exploration, and Floyd-Warshall Algorithm is used to calculate shortest paths between all pairs of metro stations.

The selected algorithm analyzes the metro graph, evaluates possible station connections, and determines the most suitable route according to the selected travel priority. The system

calculates the route path, total travel time, estimated fare, and number of line changes. It also finds alternate route options when suitable paths are available.

The user can activate the live train delay simulator by selecting a metro connection and adding a delay time. The delay simulation module updates the travel time of that connection in the graph. The routing algorithm then processes the updated metro network and recalculates the recommended route and alternate route options.

After route computation is completed, the result is returned to the web interface. The application displays the selected route, route details, travel time, fare, line changes, alternate routes, and the highlighted path on the visual metro map. The user can also access the DSA algorithm lab to compare the results of Dijkstra's Algorithm, BFS, DFS, and Floyd–Warshall Algorithm.

If the user wants to save a journey or book a ticket, the application requests user login. After successful authentication, the user can save route details, enter passenger information, select a travel date and coach preference, and confirm the booking. The ticket booking details are stored in the SQLite database, and the system displays a digital ticket with a unique booking code.

The execution process continues until the user exits the application or closes the web browser.

CHAPTER 2

ALGORITHM

2.1 Overview of the Algorithm Module

The Algorithm Module is the core component of the Smart Metro Navigation Simulator. It is responsible for processing the metro transportation network, analyzing possible travel routes, and determining the most suitable route according to the passenger's selected priority. The

project utilizes several graph-based algorithms that work together to provide efficient metro route planning, alternate route suggestions, delay handling, and network exploration.

The metro transportation network is represented using the Graph Data Structure, where every metro station is modeled as a node and every railway connection between two stations is modeled as an edge. Each edge contains important information such as travel time, ticket fare, and metro line. This representation enables efficient implementation of graph traversal and shortest-path algorithms while maintaining flexibility for adding new stations, metro lines, and route connections in future versions.

The simulator integrates four major graph algorithms: Dijkstra's Algorithm, Breadth-First Search (BFS), Depth-First Search (DFS), and Floyd-Warshall Algorithm. Dijkstra's Algorithm calculates the fastest route between two stations by considering the travel-time weights assigned to metro connections. A fare-based route calculation is used to identify the cheapest ticket route. Breadth-First Search is used to explore the metro network level by level and find routes with fewer station stops. Depth-First Search traverses the graph by exploring one path completely before backtracking, allowing users to understand metro network exploration. Floyd-Warshall Algorithm calculates shortest paths between all pairs of metro stations.

The Algorithm Module also supports live train delay simulation and alternate route generation. When a delay is added to a selected metro connection, the travel time of that edge is temporarily increased. The updated graph is then provided to the route-planning algorithms, which recalculate the fastest available route. The system also identifies alternate routes by evaluating other possible connections in the metro network.

The Algorithm Module interacts closely with the Metro Graph Management Module, User Interface Module, and Ticket Booking Module. The graph management module provides station and connection details, while the user interface module receives passenger inputs and displays the computed results. The calculated route details, including travel time, fare, station changes, and route path, can be used by the ticket booking module when an authenticated user books a metro ticket.

Overall, the Algorithm Module demonstrates the practical application of graph theory, shortest-path computation, graph traversal techniques, all-pairs path analysis, and dynamic route optimization in solving real-world metro navigation problems.

2.2 Graph Representation

The Graph Data Structure serves as the foundation of the Smart Metro Navigation Simulator. Since a metro transportation network naturally consists of stations connected by railway tracks, representing the network as a graph provides an efficient and flexible solution for metro route planning, travel-time calculation, fare comparison, and delay simulation.

In this project, each metro station is represented as a node or vertex. Examples of stations included in the simulator are Miyapur, JNTU College, KPHB Colony, Hitech City, Raidurg, Ameerpet, Secunderabad, Parade Ground, MGBS, Uppal, Nagole, Sultan Bazar, and Malakpet. Railway connections between these metro stations are represented as edges. Every edge stores information such as travel time, ticket fare, and metro line name, which are used during route computation.

The graph enables the simulator to perform different routing operations efficiently. Whenever a user requests a metro route, the graph provides the required station connectivity information to the routing algorithms. If the user activates the live train delay simulator, the travel time of a selected metro connection is temporarily increased. The graph is then processed again, allowing the algorithms to calculate a new fastest route based on the updated travel condition.

The graph implementation supports multiple neighbouring connections for each metro station, allowing the simulator to model a complex metro network with different metro lines and interchange stations. For example, Ameerpet and Parade Ground act as important interchange points connected to multiple stations and metro lines. This representation also simplifies the implementation of graph traversal algorithms such as Breadth-First Search (BFS) and Depth-First Search (DFS), as well as shortest-path algorithms such as Dijkstra's Algorithm and Floyd–Warshall Algorithm.

Using a graph-based representation offers several advantages over simple arrays or fixed route charts. It improves flexibility, supports route comparison, allows easy addition of new

metro stations and connections, and enables alternate route calculation without major changes to the application. These characteristics make the Graph Data Structure an ideal choice for developing an intelligent metro navigation and ticket-booking system.

2.3 Graph Components

The metro transportation network in the Smart Metro Navigation Simulator is modeled using a graph data structure. Instead of storing metro routes in a simple table or fixed map, the application represents the complete metro network as a collection of interconnected stations and railway connections. This design enables efficient implementation of graph algorithms while keeping the application modular, understandable, and scalable.

The graph consists of three primary components:

- **Node (Metro Station)**
- **Edge (Metro Connection)**
- **Graph (Metro Network)**

Each component performs a specific responsibility in representing the metro transportation network. Together, they provide the foundation on which all route-planning, delay-simulation, alternate-route, and ticket-booking operations are performed.

A node represents an individual metro station, such as Miyapur, Ameerpet, Secunderabad, MGBS, Uppal, or Malakpet. An edge represents a railway connection between two stations. Each edge stores important values such as travel time, ticket fare, and metro line. The graph represents the complete collection of stations and connections in the metro network.

The graph implementation follows an adjacency list representation, where each station maintains a list of all directly connected neighbouring stations. Compared to an adjacency matrix, this approach consumes less memory for a metro network with a limited number of direct connections per station. It also provides faster iteration over neighbouring stations when the routing algorithms calculate possible paths.

Whenever the user requests a metro route, the routing algorithms retrieve neighbouring stations from the graph and evaluate possible paths based on travel time, ticket fare, and

metro line changes. When the user simulates a train delay, the travel time of the selected metro connection is updated temporarily. The graph then reflects this changed condition, allowing the routing algorithms to calculate an updated route and alternate route options.

2.4 Node Class

Node class represents an individual metro station within the Smart Metro Navigation Simulator. Every metro station, such as Miyapur, JNTU College, Hitech City, Ameerpet, Secunderabad, MGBS, Uppal, and Malakpet, is represented as a separate node in the graph. The Node class acts as the fundamental building block of the metro transportation network and stores information related to a particular metro station. In the JavaScript implementation, stations are represented using station names as graph keys. Each station key stores a list of neighbouring metro connections. Conceptually, every station behaves as a node and provides the starting point for route calculation, graph traversal, map visualization, and metro connection management.

Each node stores the following information:

Attribute	Description
Station Name	Name of the metro station represented by the node, such as Ameerpet or Secunderabad.
Edges	List of metro connections linked to the current station.
Map Position	X and Y position of the station used to display it on the visual metro map.
visited	Temporary tracking value used by graph traversal algorithms such as BFS and DFS.

The `stationName` attribute uniquely identifies each metro station in the graph. It is used when users select a starting station and destination station from the route-planning interface. The routing algorithms use these station names to access neighbouring connections and calculate available travel paths.

The `edges` attribute stores all railway connections from the current station to directly connected stations. For example, Ameerpet is connected to Hitech City, Punjagutta, Secunderabad, Parade Ground, and Raidurg. This list of connections allows the routing algorithms to examine nearby stations efficiently.

The `mapPosition` attribute is used for graphical visualization. It stores the display coordinates of a station on the SVG metro map. This allows the system to draw metro stations, railway connections, and highlighted travel routes in their appropriate positions.

The `visited` attribute is used temporarily during BFS and DFS graph traversal. It helps the system identify stations that have already been explored and prevents the algorithm from repeatedly visiting the same station. This improves efficiency and avoids infinite traversal loops in the metro network.

2.5 Edge Class

2.5 Edge Class

The Edge class represents the railway connections between two metro stations in the transportation network. In graph terminology, an edge establishes a relationship between two nodes and stores the information required for route calculation. Every metro connection in the simulator is represented as an edge object within the metro graph.

Unlike a simple graph implementation that stores only the distance between two stations, the Edge class in this project contains additional information that reflects metro travel conditions. Besides the connected station, every edge maintains travel time, ticket fare, and metro line

information. These attributes enable the simulator to calculate different route options, such as the fastest route, cheapest route, and route with fewer line changes.

The Edge class stores the following information:

Attribute	Description
<code>source</code>	Starting metro station of the railway connection.
<code>station</code>	Destination metro station connected through the edge.
<code>time</code>	Normal travel time required to travel between the two stations.
<code>fare</code>	Ticket fare for the metro connection. If a separate fare is not specified, the system calculates an estimated fare using travel time.
<code>line</code>	Name of the metro line, such as Blue Line, Red Line, or Green Line.
<code>delayMinutes</code>	Temporary delay value added during live train delay simulation.

One of the important functions related to metro edges is `edgeTravelTime()`. Instead of simply returning the normal travel time, this function determines the effective travel time by considering any active delay on the selected metro connection. If a connection has a simulated train delay, the delay minutes are added to its normal travel time. This allows the routing algorithms to produce route recommendations that reflect the current simulated metro condition.

The edge information is used by Dijkstra's Algorithm, BFS, DFS, and Floyd–Warshall Algorithm during route computation. Dijkstra's Algorithm uses travel time to calculate the

fastest route, while the fare value is used to calculate the cheapest ticket route. The metro line value is used to determine the number of line changes required during a journey.

Whenever the user activates the live train delay simulator, the delay simulation module temporarily updates the effective travel time of the selected edge. The routing algorithms then calculate a new route using the updated connection information. This enables the system to provide alternate route suggestions when the primary route becomes slower because of a delay.

Overall, the Edge class acts as the communication link between metro stations and provides the routing algorithms with the information required to evaluate possible metro travel paths.

2.6 Graph Class

The Graph class is the central component of the Smart Metro Navigation Simulator. It is responsible for constructing and maintaining the complete metro transportation network. All route-planning algorithms, delay simulations, alternate-route calculations, metro map visualization, and ticket-booking operations rely on this graph structure to access station and railway connection information.

The graph is implemented using an adjacency list representation, where each metro station maintains a list of its directly connected metro edges. In the JavaScript implementation, the metro graph is stored in a `metroNetwork` object. Each station name acts as a key, and its value contains a list of neighbouring station connections. This approach is particularly efficient for metro networks because it requires less memory than an adjacency matrix while allowing fast traversal of neighbouring stations.

The Graph class maintains a collection of all metro stations and their connections. It provides the information required to identify stations by name, retrieve connected neighbouring stations, calculate travel time, calculate fare, identify metro lines, and update effective travel time during delay simulation.

The major responsibilities of the Graph class include:

- Creating the metro transportation network.

- Maintaining all metro station nodes.
- Connecting stations through railway edges.
- Storing travel time, fare, and metro line information.
- Searching for stations by their names.
- Providing neighbouring stations to routing algorithms.
- Supporting fastest-route, cheapest-route, and fewest-change calculations.
- Supporting live train delay simulation.
- Supporting alternate route generation.
- Providing station and connection information for metro map visualization.

Whenever the user initiates a route request, the routing algorithms access the graph to retrieve neighbouring metro stations and evaluate possible travel paths. Similarly, when the user simulates a train delay, the delay simulation module updates the effective travel time of the selected edge. Since the routing algorithms operate directly on the metro graph, this change is immediately reflected during route computation.

Before executing BFS or DFS, the application creates a temporary visited set to track stations that have already been explored. This prevents the algorithms from visiting the same station repeatedly and ensures efficient graph traversal without permanently changing the original metro network.

The Graph class serves as the foundation upon which all other modules operate. Without this centralized metro network representation, it would not be possible to perform fastest-route calculations, fare comparison, graph traversal, live delay simulation, alternate-route discovery, or metro map visualization efficiently.

2.7 City Builder

The Metro Network Builder module is responsible for constructing the initial metro transportation network used by the Smart Metro Navigation Simulator. Instead of manually creating metro stations and railway connections throughout different parts of the application, this module centralizes the graph creation process by defining all stations and their interconnections in a single section of the application.

During application initialization, the Metro Network Builder creates the `metroNetwork` graph object and populates it with predefined metro stations representing different locations in the virtual city. The current network includes stations such as Nizampet, Miyapur, JNTU College, KPHB Colony, Hitech City, Raidurg, Ameerpet, Secunderabad, Parade Ground, MGBS, Uppal, Nagole, Sultan Bazar, and Malakpet. After creating the station nodes, the module establishes railway connections between these stations by creating edges with appropriate travel time, ticket fare, and metro line information.

Centralizing metro graph creation within the Metro Network Builder provides several advantages. It improves code organization, simplifies future modifications to the metro network, and separates network construction from routing logic, ticket booking, delay simulation, and user interface components. If new stations, metro lines, or railway connections need to be introduced, only the Metro Network Builder requires modification, while the remaining modules continue to operate without major changes.

After the metro graph has been successfully constructed, the completed graph is used by the remaining modules of the application. The routing algorithms use this graph for fastest-route calculation, cheapest-route calculation, BFS, DFS, and Floyd–Warshall operations. The live train delay simulator uses the same graph to calculate updated travel times, while the visual metro map uses station positions and connection details to display the network.

Thus, the Metro Network Builder serves as the initialization component that prepares the virtual metro network before any route-planning, delay-simulation, journey-saving, or ticket-booking operation begins.

2.8 Dijkstra's Algorithm

Dijkstra's Algorithm is the primary routing algorithm used in the Smart Metro Navigation Simulator. Its purpose is to determine the fastest and most efficient metro route between two selected stations. Since passengers want to minimize total travel time, Dijkstra's Algorithm is used to identify the route with the lowest total travel-time cost.

The algorithm operates on the graph representation of the metro transportation network. Initially, the source station is assigned a travel time of zero, while every other station is assigned an infinite travel time. The system maintains a set of unvisited stations and repeatedly selects the station with the smallest currently known travel time.

For every selected station, the algorithm examines all directly connected neighbouring stations. If travelling through the current station provides a shorter travel time to a neighbouring station, the travel time of that neighbour is updated. The system also stores the previous station information so that the complete route can be reconstructed after the destination is reached.

Unlike a simple shortest-path implementation that uses only fixed travel time, this project also considers the live train delay simulation. When a user adds a delay to a selected metro connection, the delay minutes are added to the normal travel time of that edge. Dijkstra's Algorithm then uses the updated travel time during route computation. This ensures that the system can recommend another faster route when the original route is affected by a delay.

The algorithm continues until the destination station is reached or no unvisited station with a valid route remains. Finally, the computed fastest route is returned to the web interface, where the system displays the station path, total travel time, estimated fare, line changes, and highlighted route on the metro map.

The same Dijkstra-based approach is also adapted to calculate the cheapest ticket route. In this case, ticket fare is used as the route cost instead of travel time.

Algorithm Steps

1. Initialize the travel time of all metro stations as infinity.
2. Set the source station travel time to zero.
3. Add all metro stations to the unvisited station set.
4. Select the unvisited station with the minimum current travel time.
5. Visit all neighbouring stations connected to the selected station.
6. Calculate the new travel time through the current station.
7. Add live delay time if the selected metro connection is delayed.
8. Update the neighbouring station if a shorter route is found.

9. Store the previous station to reconstruct the route path.
10. Repeat until the destination is reached or no valid station remains.
11. Display the fastest metro route, travel time, fare, and line changes.

2.9 Breadth-First Search (BFS)

Breadth-First Search (BFS) is one of the graph traversal algorithms implemented in the Smart Metro Navigation Simulator. Unlike Dijkstra's Algorithm, which calculates the fastest weighted route based on travel time, BFS explores the metro network level by level. It is useful for finding a route with fewer station stops and for analysing connectivity between metro stations.

The algorithm begins at a selected source station and inserts it into a queue. It repeatedly removes the first station from the queue, marks it as visited, and inserts all unvisited neighbouring stations into the queue. This process continues until the destination station is found or all reachable stations have been explored.

In the project, a temporary visited set is used to prevent repeated processing of the same station. The visited set is created whenever BFS is executed, ensuring that the original metro graph remains unchanged. This avoids repeatedly visiting the same metro station and prevents unnecessary traversal loops.

Since BFS processes stations in increasing order of their distance in terms of the number of edges, it guarantees a route with the fewest station stops only when all connections are treated equally. It does not consider travel time, ticket fare, or line changes as weights. Therefore, BFS is mainly used in this project to demonstrate graph traversal, metro network connectivity, and routes with fewer station stops.

Algorithm Steps

1. Create an empty visited set.
2. Insert the source station into the queue.
3. Mark the source station as visited.

4. Remove one station from the front of the queue.
5. Check whether the current station is the destination.
6. Visit all unvisited neighbouring stations.
7. Mark neighbouring stations as visited.
8. Insert neighbouring stations into the queue.
9. Continue until the destination is found or the queue becomes empty.
10. Display the route with the number of station stops.

2.10 Depth-First Search (DFS)

Depth-First Search (DFS) is another graph traversal algorithm implemented in the Smart Metro Navigation Simulator. Unlike BFS, which explores metro stations level by level, DFS follows one complete metro path as deeply as possible before backtracking. This traversal strategy enables deep exploration of the metro network and helps users understand the structure and connectivity of different metro routes.

The algorithm starts from the selected source station and uses a stack to visit one neighbouring station at a time. Each station is marked as visited using a temporary visited set to prevent repeated processing and infinite traversal caused by cycles in the metro graph. After reaching a station with no unvisited neighbouring stations, the algorithm backtracks by removing a previous path from the stack and explores the remaining available branches.

In the Smart Metro Navigation Simulator, the original metro graph is not permanently changed during DFS execution. A temporary visited set is created whenever DFS is run. This ensures that every traversal begins with a clean state and that the same metro network can be used again for Dijkstra's Algorithm, BFS, Floyd–Warshall Algorithm, alternate-route calculation, and delay simulation.

DFS is particularly useful for exploring the connected structure of the metro network and demonstrating deep graph traversal. Although it does not guarantee the fastest route, cheapest route, or route with the fewest station stops, it provides an efficient method for analysing possible station connections and network branches.

Algorithm Steps

1. Create an empty visited set.
2. Insert the source station and its path into the stack.
3. Remove the top station path from the stack.
4. Check whether the station has already been visited.
5. Mark the current station as visited.
6. Check whether the current station is the destination.
7. Visit the unvisited neighbouring stations.
8. Insert neighbouring station paths into the stack.
9. Backtrack when no unvisited neighbouring station is available.
10. Continue until the destination is found or the stack becomes empty.

2.11 Priority Queue

A priority queue is an important data structure commonly used in shortest-path algorithms such as Dijkstra's Algorithm. Unlike a normal queue that follows the First-In-First-Out (FIFO) principle, a priority queue removes the element with the highest priority first. In route-planning problems, priority is generally determined by the smallest current travel cost.

In the Smart Metro Navigation Simulator, Dijkstra's Algorithm follows the same priority-based route-selection concept. The system maintains the current travel time for each metro station and repeatedly selects the unvisited station with the minimum known travel time. This station is then processed before stations with greater travel-time values.

Whenever a shorter route to a neighbouring station is discovered, the system updates the travel-time value of that station. The updated value is then considered during the next station-selection step. This process continues until the destination station is reached or no valid route remains.

The current JavaScript implementation uses an unvisited station set and selects the station with the minimum current travel time during each iteration. This performs the same logical purpose as a priority queue for the current metro network. For a much larger metro network, a dedicated priority queue or min-heap can be used to improve route-calculation efficiency further.

The priority-based selection process is important because it ensures that stations with lower travel time are processed first. This helps Dijkstra's Algorithm calculate the fastest metro route efficiently, even when multiple possible connections and live train delays are present in the network.

Overall, priority-based route selection plays an important role in optimizing fastest-route calculation and ensuring that metro passengers receive an efficient travel recommendation.

2.12 Live Train Delay Simulator

The Live Train Delay Simulator is responsible for simulating train delays within the Smart Metro Navigation Simulator. It acts as the central component that modifies the effective travel time of a selected metro connection whenever a delay is added by the user.

When a user selects a metro connection and chooses a delay value, the Live Train Delay Simulator identifies the affected connection using its source and destination stations. The selected delay minutes are added to the normal travel time of that metro edge. For example, if a connection normally takes 5 minutes and a delay of 10 minutes is applied, the effective travel time for that connection becomes 15 minutes.

After the metro connection has been updated, the routing algorithms automatically operate using the changed travel-time value. As a result, the fastest route is recalculated without requiring the user to manually search for another route. If the original route becomes slower because of the delay, the system may recommend a different route or display alternate route options.

The Live Train Delay Simulator also coordinates with the graphical user interface so that users can immediately observe the effect of the delay on metro journey planning. The delayed metro connection is displayed as an orange dashed line on the visual metro map, while the updated route, travel time, and alternate routes are shown in the route result section.

By separating delay simulation from the main routing logic, the application maintains a modular design that is easier to maintain and extend. The delay module communicates with the metro graph and algorithm modules without permanently changing the original metro network data.

Overall, the Live Train Delay Simulator serves as the bridge between dynamic metro conditions and graph-based route computation. It demonstrates how intelligent transportation systems can adapt route recommendations when train delays affect passenger journeys.

2.13 Train Delay Simulator

The Train Delay Simulator provides an interactive environment in which metro delay scenarios are executed. It is responsible for allowing users to select a metro connection, apply a delay value, observe the effect on the metro network, and compare route recommendations before and after the delay.

The simulator works together with the Metro Graph Management Module and Live Train Delay Module. Initially, the metro transportation network is available in its normal state, where every connection has its predefined travel time. When the user selects a metro connection and adds a delay, the simulator invokes the Live Train Delay Module to increase the effective travel time of the affected connection.

Once the effective travel time has been updated, the routing algorithms execute again to determine the fastest available metro route. The results are immediately displayed through the web interface, allowing users to compare the updated journey time, recommended route, and alternate route options. The delayed connection is also highlighted on the metro map using an orange dashed line.

This simulation-based approach provides a practical demonstration of how graph algorithms can be applied to solve real-world smart transportation problems. Users can experiment with different delay values and different metro connections to observe how changes in train availability influence route-planning decisions.

The Train Delay Simulator therefore transforms the project from a simple metro graph application into an interactive educational tool for route optimization, alternate-route discovery, and dynamic passenger journey planning.

2.14 Graphical User Interface (MainFrame)

The Web User Interface provides the graphical interface of the Smart Metro Navigation Simulator. It serves as the primary point of interaction between the user and the application, allowing users to perform metro route planning, delay simulation, user authentication, and ticket-booking operations through an intuitive web interface.

The graphical interface enables users to select starting and destination stations, choose a travel priority, execute route-planning algorithms, simulate train delays, view alternate routes, and see the calculated metro route on the visual metro map. Instead of interacting with the program through command-line inputs, users can perform these operations using station selection menus, buttons, sliders, forms, route cards, and map components.

The web interface is developed using HTML, CSS, and JavaScript. HTML defines the structure of the application, including the route planner, login portal, delay simulator, ticket-booking form, and digital ticket display. CSS provides colours, responsive layout, metro line styling, buttons, cards, and mobile-friendly design. JavaScript handles user actions, sends requests to the server, executes route calculations, updates the metro map, and displays the results dynamically.

The user interface communicates with the Metro Graph Management Module, Algorithm Module, Live Train Delay Simulator, User Authentication Module, and Ticket Booking Module. User actions are translated into JavaScript function calls and API requests, after which the results are displayed on the interface. This separation between the user interface and application logic improves maintainability and makes the system easier to extend.

The web interface also includes user registration, login, forgot-password functionality using email OTP, saved journey management, ticket booking, coach preference selection, ticket cancellation, and digital ticket display. These features provide a complete passenger-oriented experience in addition to metro route planning.

Overall, the graphical user interface enhances usability and makes the Smart Metro Navigation Simulator suitable for educational demonstrations, interactive graph-algorithm learning, and smart transportation route-planning simulations.

2.15 Time Complexity Analysis

The Smart Metro Navigation Simulator implements several graph algorithms, each having its own computational complexity. Understanding the time complexity helps evaluate the efficiency of the application when operating on larger metro transportation networks.

Algorithm / Operation	Time Complexity
Metro Graph Construction	$O(V + E)$
Dijkstra's Algorithm	$O(V^2 + E)$
Breadth-First Search (BFS)	$O(V + E)$
Depth-First Search (DFS)	$O(V + E)$
Floyd–Warshall Algorithm	$O(V^3)$

Adding a Metro Station $O(1)$

Adding a Metro Connection $O(1)$

Live Train Delay Update $O(1)$

Alternate Route Generation $O(L \times (V^2 + E))$

Where:

- V = Number of metro stations (vertices).
- E = Number of metro connections (edges).
- L = Number of connections in the main calculated route.

The metro network uses an adjacency list representation, which improves the performance of graph traversal algorithms because the system only processes directly connected neighbouring stations. This makes BFS and DFS efficient for sparse metro networks.

The current Dijkstra's Algorithm implementation uses an unvisited station set and selects the station with the minimum travel-time value during each iteration. Therefore, its time complexity is $O(V^2 + E)$. For

a very large real-world metro network, a min-heap or priority queue implementation can be used to improve Dijkstra's Algorithm to $O((V + E) \log V)$.

Floyd–Warshall Algorithm has a higher time complexity of $O(V^3)$ because it checks possible paths between every pair of metro stations through every intermediate station. It is suitable for the current simulator because the metro network contains a limited number of stations. However, Dijkstra's Algorithm is more suitable for individual route queries in a large metro network.

Overall, the time complexity of the application is suitable for the current virtual metro network and provides efficient route planning,

2.16 Space Complexity Analysis

The Smart Metro Navigation Simulator stores the metro transportation network using an adjacency list representation. This representation requires memory proportional to the number of metro stations and railway connections. It is more memory-efficient than an adjacency matrix, particularly when each metro station is connected to only a limited number of neighbouring stations.

The overall space complexity of the application is determined by the storage required for metro stations, metro connections, auxiliary data structures, route paths, and algorithm-specific variables.

Component	Space Complexity
Metro Graph Storage	$O(V + E)$
Dijkstra's Algorithm	$O(V)$
BFS Visited Set and Queue	$O(V)$
DFS Visited Set and Stack	$O(V)$
Floyd–Warshall Algorithm	$O(V^2)$
Live Train Delay Data	$O(1)$
Metro Map Station Positions	$O(V)$
Alternate Route Storage	$O(V)$

Where:

- V = Number of metro stations.
- E = Number of metro connections.

The metro graph stores each station and its directly connected neighbouring stations. Therefore, the graph storage requires $O(V + E)$ memory. This is efficient because the application does not need to store unnecessary information for station pairs that are not directly connected.

Dijkstra's Algorithm stores travel-time values, previous-station information, and an unvisited station set. These structures require $O(V)$ space. BFS and DFS also use a visited set and either a queue or stack, requiring $O(V)$ space for graph traversal.

Floyd–Warshall Algorithm requires $O(V^2)$ space because it stores distance values and next-station information for every pair of metro stations. Although this uses more memory than the other algorithms, it is suitable for the current simulator because the metro network contains a limited number of stations.

The live train delay simulator requires only $O(1)$ additional space because it stores the selected delayed connection and delay minutes. The metro map also stores display positions for each station, requiring $O(V)$ space.

Overall, the adjacency list implementation minimizes unnecessary memory usage while allowing efficient route planning, graph traversal, delay simulation, alternate route calculation, and metro map visualization.

CHAPTER 3

RESULTS

3.1 Introduction

The Smart Metro Navigation Simulator was successfully developed as a web-based application using HTML, CSS, JavaScript, Node.js, SQLite, and graph-based routing algorithms. The application integrates a metro graph data structure with shortest-path

algorithms, graph traversal algorithms, user authentication, train delay simulation, alternate route generation, and ticket-booking functionality. The implemented system demonstrates how Data Structures and Algorithms can assist passengers in selecting efficient metro routes based on travel time, ticket fare, station stops, and line changes.

The application was tested using different metro journey scenarios by selecting various source and destination stations, changing the route priority, and introducing delays on selected metro connections. In each case, the simulator correctly processed the metro network and displayed route information. The system calculated fastest routes using Dijkstra's Algorithm, displayed route options based on fare and station changes, and recalculated the fastest route after a train delay was simulated.

The simulator also supports user registration, login, saved journeys, ticket booking, coach preference selection, ticket cancellation, and password reset through email OTP authentication. The results confirm that the system functions as intended and effectively demonstrates the practical application of graph algorithms, web development, and database management in smart metro transportation.

3.2 Functional Results

The completed application successfully performs all major operations required for metro route planning and passenger management. The developed system provides an interactive web interface through which users can plan metro journeys, compare routes, simulate delays, save journey details, and book digital metro tickets.

The primary functionalities verified during testing include:

- Creation of the metro transportation network using graph data structures.
- Representation of metro stations as graph nodes.
- Representation of railway connections as weighted graph edges.
- Storage of travel time, ticket fare, and metro line details for each connection.
- Fastest route calculation using Dijkstra's Algorithm.

- Cheapest ticket route calculation using fare as the route cost.
- Route calculation with fewer station stops using Breadth-First Search (BFS).
- Metro network exploration using Depth-First Search (DFS).
- All-pairs shortest-path computation using Floyd–Warshall Algorithm.
- Visual display of metro stations and railway connections on an interactive metro map.
- Display of travel time, estimated fare, and number of line changes.
- Generation of alternate routes for the selected journey.
- Live train delay simulation by increasing the travel time of a selected metro connection.
- Dynamic recalculation of the fastest route after a simulated delay.
- User registration, login, logout, and session management.
- Password reset using six-digit email OTP authentication.
- Saving planned journeys in the SQLite database.
- Metro ticket booking with passenger name, travel date, and coach preference.
- Digital ticket generation with a unique booking code.
- Ticket history display and ticket cancellation.

Testing confirmed that the modules interacted correctly with one another and produced reliable route-planning, authentication, booking, and delay-simulation results.

3.3 Algorithm Performance

The graph algorithms implemented in the Smart Metro Navigation Simulator performed efficiently during testing. Dijkstra’s Algorithm consistently identified the fastest available metro route based on the total travel time between the selected stations. When the user simulated a delay on a metro connection, the algorithm used the updated travel-time value and recalculated the route. This demonstrated the ability of graph-based routing to adapt to changing metro conditions.

The fare-based route calculation successfully identified the cheapest route by using ticket fare as the route cost. This allowed passengers to compare the fastest route with the lowest-cost

route. In some cases, the cheapest route required more travel time, while the fastest route required a higher fare. This comparison demonstrated how the same graph can provide different route recommendations according to different passenger requirements.

Breadth-First Search successfully explored metro stations level by level and produced routes with fewer station stops. Depth-First Search successfully traversed the metro network by exploring one branch deeply before backtracking. Although DFS does not guarantee the fastest route, it was useful for demonstrating graph exploration and metro network connectivity.

Floyd–Warshall Algorithm successfully calculated shortest paths between all pairs of metro stations. It was useful for demonstrating all-pairs path computation in the DSA Algorithm Lab. Since the current simulator contains a limited number of metro stations, Floyd–Warshall Algorithm performed effectively for the virtual metro network.

The adjacency list representation improved the performance of graph traversal and route-planning operations because the algorithms processed only directly connected neighbouring stations. The current Dijkstra implementation uses an unvisited station set for route-cost selection. For a much larger real-world metro network, a min-heap or priority queue can be added to improve shortest-path performance further.

3.4 Observations

Several important observations were made during the development and testing of the Smart Metro Navigation Simulator.

The graph representation proved to be highly suitable for modelling a metro transportation network because it accurately represents metro stations and railway connections. The adjacency list implementation reduced unnecessary memory usage and simplified graph traversal operations.

Dijkstra's Algorithm produced the fastest metro route according to travel-time values. The fare-based route calculation demonstrated that the fastest route and cheapest ticket route may be different. This showed the importance of allowing passengers to select a route based on their personal travel preference.

The live train delay simulation feature enabled the system to represent changing metro conditions. Adding a delay to a selected connection immediately affected the travel time of the route. The system recalculated the fastest route and displayed alternate route options, demonstrating the adaptability of graph-based route-planning techniques.

The visual metro map improved the usability of the application. Users could see the complete metro network, highlighted route, active stations, delayed connection, and metro line colours. This helped users understand the route visually instead of depending only on station names and text-based results.

The user authentication and database modules successfully stored user accounts, saved journeys, password-reset OTP records, and ticket-booking details. The ticket module generated a digital ticket with passenger information, journey route, fare, travel date, coach preference, booking code, and ticket status.

The modular application design simplified implementation and testing. Individual components such as the metro graph, algorithms, delay simulator, authentication module, database module, ticket-booking module, and web interface could be modified independently without affecting the remaining modules.

The project effectively demonstrates how classical graph algorithms can be integrated with modern web technologies to create a practical smart transportation application.

3.5 Advantages of the Developed System

The developed Smart Metro Navigation Simulator provides several advantages over traditional static metro maps and manual route-planning methods.

- Graph-based modelling accurately represents metro stations and railway connections.
- The system provides fastest, cheapest, and fewer-stop route options.
- Dijkstra's Algorithm efficiently calculates the fastest available route.
- BFS and DFS demonstrate graph traversal techniques for metro network analysis.
- Floyd–Warshall Algorithm demonstrates all-pairs shortest-path calculation.
- Live train delay simulation enables dynamic route recalculation.
- Alternate route suggestions help passengers select another route when delays occur.
- The visual metro map makes route information easy to understand.
- The web-based interface is simple, interactive, and accessible through a browser.
- User registration and login enable personalized journey and ticket management.
- Password reset through email OTP improves account security.
- SQLite database support enables storage of user details, saved journeys, and tickets.
- Ticket booking supports passenger details, travel date, coach preference, booking code, and cancellation.
- The modular design improves maintainability and future scalability.
- The simulator serves as an educational platform for understanding graph algorithms and smart transportation concepts.
- New metro stations, connections, lines, and advanced features can be added easily in future versions.
-

3.6 Limitations

Although the Smart Metro Navigation Simulator successfully demonstrates graph-based metro route planning, several limitations remain.

The metro network is predefined and uses simulated station connections, travel times, and fares. It does not currently use official real metro data or real geographical distances. Therefore, the application should be treated as an educational simulator and not as an official passenger navigation system.

The live train delay feature is manually simulated by the user. The system does not receive real-time train positions, live delay information, passenger crowd data, station congestion details, or official metro service alerts.

The ticket-booking functionality is a local simulation. It stores ticket records in the SQLite database and displays a digital ticket, but it does not support real payment gateway integration, official QR-code validation, turnstile access, actual seat assignment, or direct integration with a metro authority.

The password reset functionality requires Gmail SMTP configuration and a Gmail App Password to send real OTP emails. Without valid Gmail configuration, the OTP email delivery feature remains unavailable.

The application is intended primarily for educational purposes and does not currently support GPS navigation, multilingual support, accessibility features for all passenger groups, artificial intelligence-based crowd prediction, real-time demand forecasting, or deployment on a public cloud server.

These limitations provide opportunities for future enhancement and real-world integration.

3.7 Chapter Summary

The implementation and testing of the Smart Metro Navigation Simulator demonstrate the successful application of graph data structures, graph algorithms, web development, user authentication, database management, and digital ticket booking in metro route planning.

The system correctly performs fastest-route calculation, cheapest-route comparison, BFS and DFS traversal, Floyd–Warshall all-pairs path computation, live train delay simulation, alternate-route generation, user login, password reset through email OTP, saved journey storage, ticket booking, and ticket cancellation. The interactive metro map and web interface provide a user-friendly environment for visualizing routes and understanding route-planning results.

The experimental results confirm that the integration of Dijkstra's Algorithm, Breadth-First Search, Depth-First Search, Floyd–Warshall Algorithm, and graph data structures enables efficient metro route planning within a simulated transportation network. Overall, the project achieves its objectives and provides a practical demonstration of how Data Structures and Algorithms can be applied to solve smart transportation and metro passenger-management problems.

CHAPTER 4

CONCLUSION

4.1 Conclusion

The Smart Metro Navigation Simulator successfully demonstrates the practical application of Data Structures and Algorithms in solving metro route-planning and passenger-management problems. The project integrates graph theory, shortest-path computation, graph traversal techniques, web development, database management, user authentication, and ticket booking to simulate an intelligent metro navigation system.

The metro transportation network was modelled using an adjacency list representation, where metro stations are represented as nodes and railway connections are represented as weighted edges. Each edge stores travel time, ticket fare, and metro line information. This representation enabled efficient implementation of graph algorithms while minimizing unnecessary memory usage.

The use of Dijkstra's Algorithm allowed the simulator to determine the fastest metro route between two selected stations. Breadth-First Search (BFS) was used to find routes with fewer station stops, while Depth-First Search (DFS) was used for metro network exploration. Floyd-Warshall Algorithm was implemented to calculate shortest paths between all pairs of metro stations. The system also provided a fare-based route calculation for identifying the cheapest ticket route.

One of the major achievements of the project is its ability to dynamically respond to simulated train delays. When a delay is added to a selected metro connection, the effective travel time is updated and the system automatically recalculates the fastest route. The application also displays alternate route options, allowing passengers to choose a suitable journey when the original route becomes slower.

The project also includes a complete passenger-management system. Users can register, log in, reset passwords through email OTP authentication, save journeys, book tickets, select coach preferences, view digital tickets, and cancel booked tickets. SQLite database technology is used to store user information, journey records, OTP details, and ticket bookings securely.

The modular architecture adopted in the project improves readability, maintainability, and future extensibility. Individual modules such as Metro Graph Management, Algorithm Module, Live Train Delay Simulator, User Authentication Module, Database Module, Ticket Booking Module, and Web User Interface perform clearly defined responsibilities while working together to achieve the overall objectives of the system.

Overall, the project successfully fulfills its intended objectives by demonstrating how graph-based algorithms can be applied to intelligent metro route planning. In addition to strengthening programming and problem-solving skills, the project provides valuable practical experience in implementing Data Structures and Algorithms within a realistic smart transportation software application.

4.2 Future Scope

Although the current implementation successfully demonstrates metro route planning and passenger management, several enhancements can further improve the capabilities of the system.

Future versions of the application may integrate official real-time metro data, including live train locations, arrival times, service alerts, station crowd levels, and actual delay notifications. Incorporating these updates would allow the routing algorithms to generate more accurate journey recommendations based on current metro conditions.

The application may also be extended to support a larger metro network with official station names, actual metro lines, geographical coordinates, and real ticket-fare rules. Integration with mapping services and Geographic Information Systems (GIS) would enable the system to display a realistic city map instead of a simulated metro map.

Additional algorithms such as A* Search, Bellman–Ford Algorithm, Yen’s K-Shortest Paths Algorithm, and advanced minimum-change route algorithms can be incorporated to compare different route-planning strategies. Machine Learning techniques may also be introduced to predict passenger crowd levels, train delays, peak-hour congestion, and travel demand.

The ticket-booking system can be enhanced by integrating an online payment gateway, QR-code ticket validation, actual seat or coach allocation, ticket download as PDF, refund handling, and notifications through email or SMS. Multi-language support, accessibility features, voice guidance, and mobile application integration can improve usability for a wider range of passengers.

Cloud-based database deployment, role-based administration, metro staff dashboards, passenger analytics, and security monitoring can further improve the practical usefulness of the system. These enhancements would transform the simulator into a more comprehensive smart metro navigation platform for public transportation management, academic learning, and passenger assistance.

REFERENCES

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms* (4th Edition). MIT Press, 2022.
2. Sedgewick, R., & Wayne, K. *Algorithms* (4th Edition). Addison-Wesley, 2011.
3. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. *Data Structures and Algorithms in Java* (6th Edition). Wiley, 2014.

4. Dijkstra, E. W. “A Note on Two Problems in Connexion with Graphs.” *Numerische Mathematik*, 1, 269–271, 1959.
5. Mozilla Developer Network. [JavaScript Guide](#). Accessed July 2026.
6. Mozilla Developer Network. [HTML: HyperText Markup Language](#). Accessed July 2026.
7. Mozilla Developer Network. [CSS: Cascading Style Sheets](#). Accessed July 2026.
8. Node.js. [Node.js Documentation](#). Accessed July 2026.
9. Node.js. [SQLite API Documentation](#). Accessed July 2026.
10. SQLite. [SQLite Documentation](#). Accessed July 2026.
11. Nodemailer. [Nodemailer Documentation](#). Accessed July 2026.
12. GeeksforGeeks. [Graph Data Structure and Algorithms](#). Accessed July 2026.

Supporting Imagery:

The screenshot shows the 'Smart Metro Navigator' web application. At the top, it says 'METRO ROUTE PLANNER' and 'Smart Metro Navigator'. Below the title, it says 'Find the quickest way across the city.' The interface is divided into two main sections. The first section is 'YOUR ACCOUNT' with a 'Log in to save journeys' prompt. It includes a 'Log in' button, a 'Create account' button, and input fields for 'Full name', 'Email', and 'Password'. There is also a 'Log in' button and a 'Forgot password?' link. The second section is for route planning. It has 'Starting station' (Miyapur) and 'Destination station' (Nizampet) dropdowns. Below these are three buttons: 'Fastest' (selected), 'Cheapest', and 'Fewest changes'. There is a 'Live train delay simulator' section with a dropdown for 'Miyapur ↔ JNTU College' and a slider. At the bottom is a large blue button labeled 'Find fastest route'.

The screenshot shows the 'Your fastest route' section. It displays a sequence of stations: Miyapur → JNTU College → KPHB Colony → Hitech City → Ameerpet → Secunderabad → Parade Ground → MGBS → Sultan Bazar. Below this, it shows 'Travel time: 39 minutes', 'Estimated fare: ₹78', and 'Line changes: 2'. There are two buttons: 'Log in to save this journey' and 'Log in to book ticket'. Below this is the 'Alternate routes' section, which says 'Different paths, ranked using your selected priority.' It lists two alternatives: 'ALTERNATIVE 1' (40 min · ₹68 · 2 changes) and 'ALTERNATIVE 2' (43 min · ₹96 · 1 changes). At the bottom is a 'LEARN BY TRYING' section titled 'DSA algorithm lab' with a description: 'Use your selected stations to compare four graph algorithms.' It has four buttons: 'Dijkstra' (selected), 'BFS', 'DFS', and 'Floyd-Warshall'. Below this is a section titled 'Dijkstra — fastest total travel time' with the text: 'It compares the total number of minutes for possible routes and keeps improving the best known route.'

